Edit


# Track Local Changes and Manage Commits

## Challenge Overview

### Understand the scenario

You are a Developer for Hexelo, an organization that needs to practice staging, committing, and viewing changes.

In this Challenge Lab, you will stage, commit, view, and undo changes with full control over your project history. First, you will create new files and track them with git add, and then you will write descriptive commit messages. Next, you will modify files and view diffs, and then you will undo changes by using git restore. You will view commit history with flags (--oneline, --graph, --decorate, and --all). Finally, you will tag your most recent commits, stash and reapply changes, and push your project to your GitHub repository.

### Navigating the Challenge Lab

Unable to process content from https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md

▶ Quick tips for navigating the Challenge Lab instructions.

New to Challenge Labs? Click here to learn more.

# Configure your environment and starter project

## Hints Enabled

No Yes

- Sign in to the virtual machine as **LabUser** using T Passw0rd! as the password.
- Enter your GitHub Username, Email Address, Password, and GitHub Account URL into the following associated textboxes:

  **GitHub Username**

  **GitHub Email Address**

  **GitHub Password**

  **GitHub URL**

- Open **Microsoft Edge**, go to T https://github.com, and then sign in as T <Name> using T <GithubPW> as the password.

  **Expand this hint for guidance on signing in to a GitHub account.**

  - On the Windows taskbar, in Search, enter **Edge**, and then select **Microsoft Edge** to open a new browser.
  - In the Address bar, enter T https://github.com, and then press **Enter**.
  - On the GitHub home page, in the upper right-hand corner, select **Sign in**.
  - On the *Sign in to GitHub* page, in Username or email address, sign in as T <Name> using T <GithubPW> as the password.
  - If presented with the *Save password* dialog, select **Never**.
  - On the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.

- Open a new **Microsoft Edge** browser tab, go to T https://git-scm.com/download/win, download and install the latest version of **Git** by using the default options, and then launch **Git Bash**.

  **Expand this hint for guidance on installing Git.**

  - In Microsoft Edge, open a new tab.
  - In the Address bar, enter T https://git-scm.com/download/win, and then press **Enter**.
  - On the **git** Downloads page, in *Downloads for Windows*, select **Click here to download**.
  - In the *Downloads notification* dialog, select **Open file**.
  - In the *User Account Control - Do you want to allow this app to make changes to your device?* dialog, select **Yes**.
  - On the Git Setup page, ensure that *Only show new options* is selected, select **Next** (if necessary), and then select **Install**.
  - On the *Completing the Git Setup Wizard* page, select the **Launch Git Bash** checkbox, and then select **Finish**.
  - If prompted, in the *Select an app to open this .html file* dialog, select **Always**, and then review and close the **Release Notes** browser tab.

Want to learn more? Review the documentation on installing git.

- Verify the installation by running the following command in the **Git Bash** terminal:

```
git --version
```

The results should display *git version 2.52.0.windows.1* or later.

Always verify Git is installed before running Git Bash commands.

Want to learn more? Review the documentation on the Git Bash terminal.

- Set your Git Identity by using the T git config command using and your GitHub **username** and **email address**.

**Expand this hint for guidance on setting your Git identity.**

- In the Git Bash Terminal, run the following commands to set your Git identity:

```
git config --global user.name "<Name>"
```

```
git config --global user.email "<GithubUN>"
```

- The results should look like the following screenshot:


Want to learn more? Review the documentation on setting your Git Identity.

The *--global* Git identity can be changed as many times as you like (each change overwrites the previous one), and you can always override it for individual repositories using *--local*.

- Confirm your settings by using the T git config command.

**Expand this hint for guidance on confirming your settings.**

- Run the following command to confirm your settings:

```
git config --list
```

Want to learn more? Review the documentation on confirming your settings.

## Create a local repository

- In the Git Bash terminal, create a project folder named T lab-git-intro{LAB_INSTANCE_ID} in the T D:\LabFiles folder and change to the new folder by running the following command:

```
mkdir /d/LabFiles/lab-git-intro{LAB_INSTANCE_ID} && cd /d/LabFiles/lab-git-int
```

Want to learn more? Review the following command documentation:

- cd
- mkdir

- Initialize the Git repository by running the following command:

```
git init
```

The results should look like the following screenshot:


Want to learn more? Review the documentation on initializing a Git repo.

The .git folder inside a repository is where Git stores all its metadata–including your commit history, branches, and configuration. Deleting this folder will "un-Git" your project.

- Rename your branch from *master* to T main by running the following T git branch command:

```
git branch -M main
```

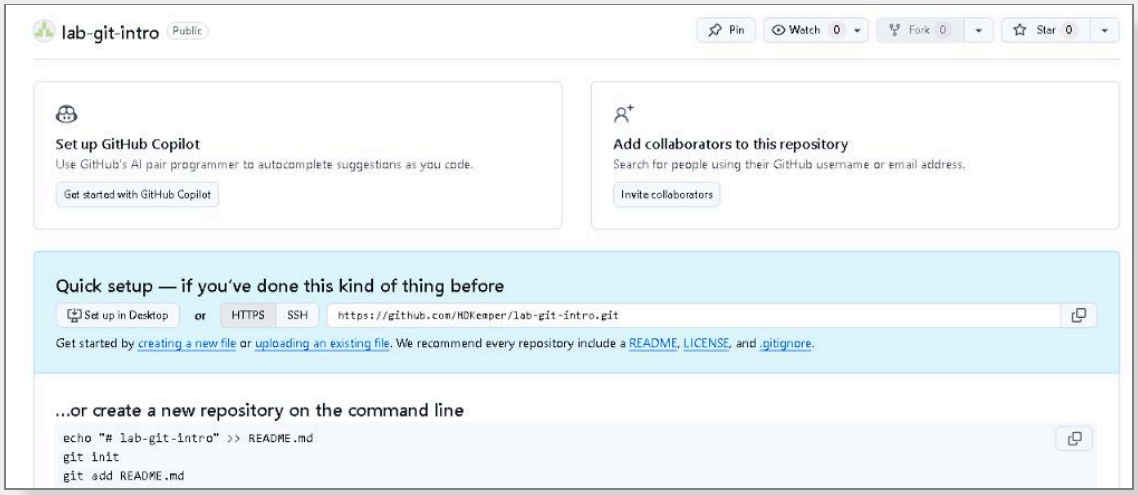
Want to learn more? Review the documentation on renaming a branch.

## Create a GitHub repository

- Switch to the **GitHub** browser tab, and then if needed, sign in as T <Name> using T <GithubPW> as the password.
- Create a GitHub repository named T lab-git-intro{LAB_INSTANCE_ID}.

**Expand this hint for guidance on creating a GitHub repository.**

- On the GitHub Dashboard Home page, in Create your first project, select **Create repository**.
- On the New repository page, in Create a new repository, in Repository name, enter T lab-git-intro{LAB_INSTANCE_ID}, and then select **Create repository**.
- Your new repository page should look like the following screenshot:




Repositories on GitHub can be public or private. Public repos are visible to everyone, while private repos restrict visibility to collaborators you choose–great for proprietary or in-progress work.

Want to learn more? Review the documentation on creating a GitHub repository.

- In the Git Bash Terminal, connect your local repository to your remote repository using the URL T <GithubURL>/lab-git-intro{LAB_INSTANCE_ID}.git and by running the following T git remote command:

```
git remote add origin <GithubURL>/lab-git-intro{LAB_INSTANCE_ID}.git
```

Want to learn more? Review the documentation on the git remote command.

## Check your work

Verify

# Stage and save changes

## Hints Enabled

No Yes

- Create a file named T hello.txt that contains T Hello, Git World! as text by running the following command:

```
echo "Hello, Git World!" > hello.txt
```

Want to learn more? Review the documentation on the echo command.

- Run the following command to check the Git status:

```
git status
```

Output should display hello.txt as **Untracked**.

Want to learn more? Review the documentation on the git status command.

- Stage the T hello.txt file by using the T git add command.

**Expand this hint for guidance on staging a file.**

- In your Git Bash terminal, run the following command to stage the file:

```
git add hello.txt
```

Want to learn more? Review the documentation on staging a file.

Staging lets you control what goes into your next commit. This is ideal for separating concerns.

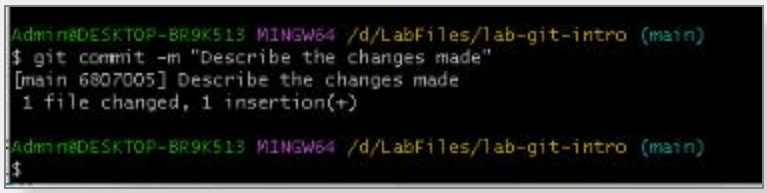
- Commit changes by using the T git commit command and T "Initial commit: added hello.txt" as the message.

**Expand this hint for guidance on committing staged changes with a message.**

- Run the following command to commit changes with a message:

```
git commit -m "Initial commit: added hello.txt"
```

- The results should look like the following screenshot:




A good commit message should tell *what* changed and *why*. This makes collaboration easier. Use a clear and descriptive message (e.g., "Fix header formatting in index.html").

Want to learn more? Review the documentation on the git commit command.

- Modify the **hello.txt** file by adding T And then goodbye! to line 2.

**Expand this hint for guidance on modifying a file.**

- Open **File Manager** by using the Windows taskbar button.
- Browse to the **D:\LabFiles\lab-git-intro{LAB_INSTANCE_ID}** folder, open the **hello.txt** file in **Notepad**, add T And then goodbye! to line 2, and then select **File > Save**.

Close **Notepad** and **Windows Explorer**.

Want to learn more? Review the documentation on reviewing changes between files.

- Review the differences between the working directory and the last commit by using the T git diff command.

**Expand this hint for guidance on reviewing changes between files.**

- In the Git Bash terminal, run the following command to review the changes:

```
git diff
```

- The results should look like the following screenshot:

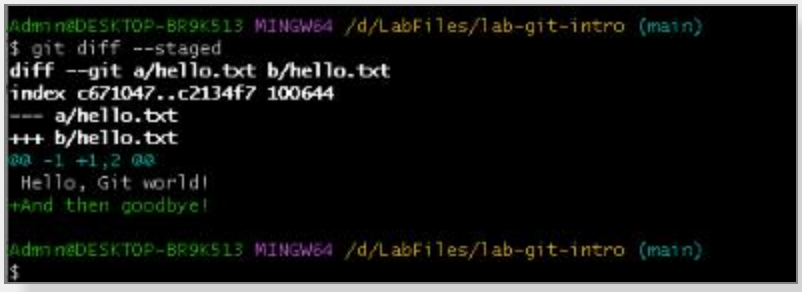

- Stage the T hello.txt file by using the T git add command.
- Preview the T staged changes by using the T git diff command.

**Expand this hint for guidance on previewing staged changes.**

- Run the following command to preview staged changes:

```
git diff --staged
```

- The results should look like the following screenshot:

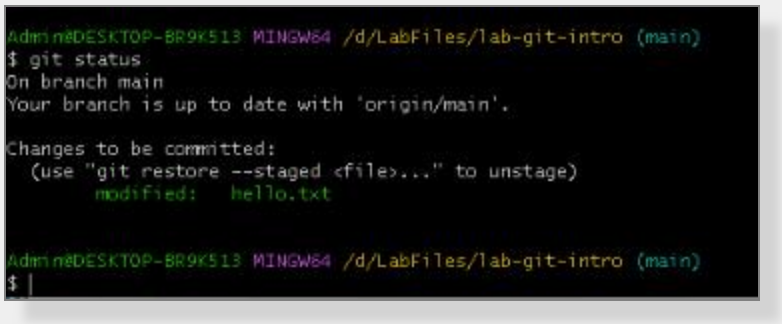



- Verify the modified files by using the T git status command.

**Expand this hint for guidance on the git status command.**

```
git status
```

The results should look like the following screenshot:




Want to learn more? Review the documentation on verifying modified files.

- Stage all modified, new, and deleted files in this folder (and its subfolders) by using the T git add command.

**Expand this hint for guidance on staging files.**

- Run the following command to stage all modified, new, and deleted files:

```
git add .
```

When you run *git add .* (or git add --all), Git only stages files that are currently different between your working directory and the last commit. Running *git add .* after already adding hello.txt does nothing for that file because its changes are already staged–but the command will still stage any other modified or new files that haven't been added yet.

- Commit changes by using the T git commit command and T "Describe the changes made" as the message.

**Expand this hint for guidance on committing staged changes with a message.**

- Run the following command to commit changes with a message:

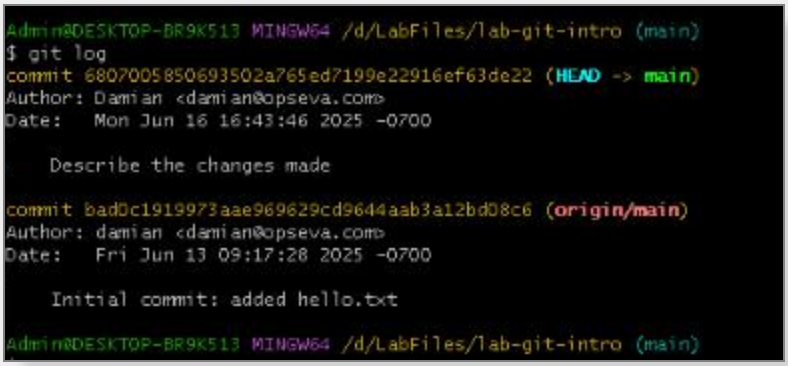
```
git commit -m "Describe the changes made"
```

A good commit message should tell *what* changed and *why*. This makes collaboration easier. Use a clear and descriptive message (e.g., "Fix header formatting in index.html").

- Display the commit history by using the following command:

```
git log
```

**The results should look like the following screenshot:**




Want to learn more? Review the documentation on displaying the commit history.

Use *--graph* and *--oneline* together to get a quick yet informative overview of your project history.

- Display a visual history by using the T git log command.

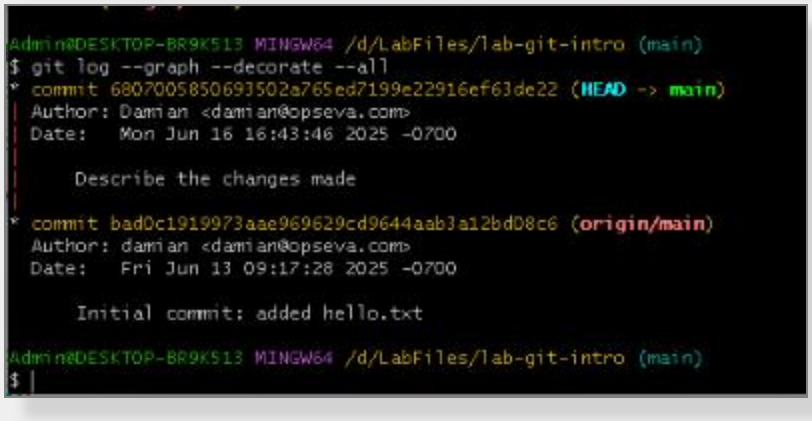
**Expand this hint for guidance on displaying a visual history.**

- Run the following commands to display a visual history:

```
git log --oneline
```

```
git log --graph --decorate --all
```

- The results should look like the following screenshots:


## Check your work

Verify

# Reverse changes and amend commits

## Hints Enabled

No Yes

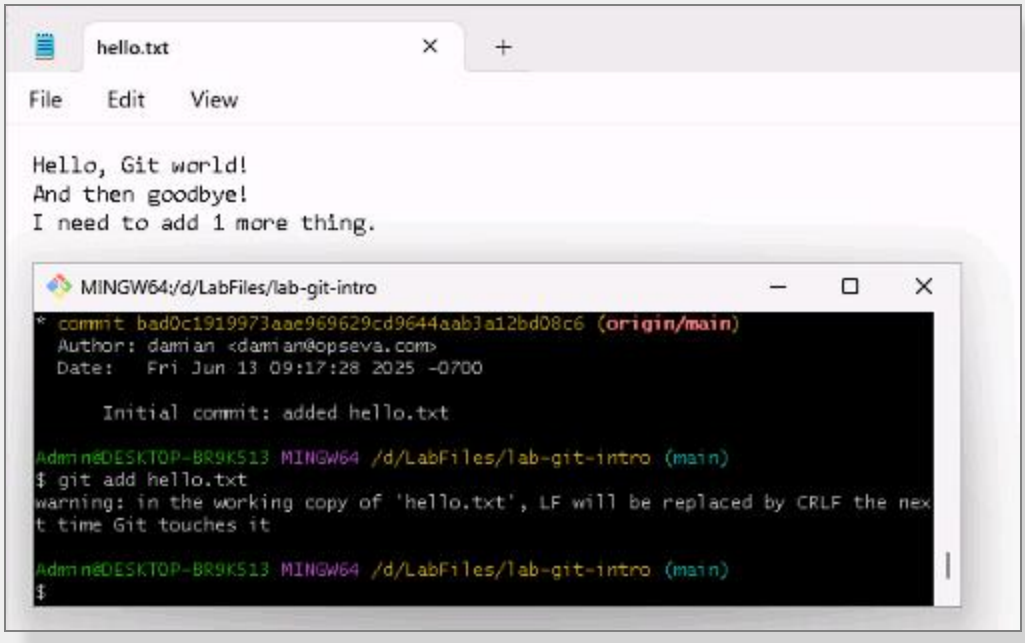
- Open **hello.txt**, add T I need to add 1 more thing on a new line, and then save the file.
- Stage the T hello.txt file by using the T git add command.

**Expand this hint for guidance on staging a file.**

- Run the following command to stage the file:

```
git add hello.txt
```

- The results should look like the following screenshot:




Use *git status* to confirm that the hello.txt file is in a staged state. In the Git Bash Terminal, you will see:

Changes to be committed:

modified: hello.txt

- Unstage the changes to the T hello.txt file by using the T git restore command with the T --staged flag.

**Expand this hint for guidance on unstaging changes.**

- Run the following command to unstage changes:

```
git restore --staged hello.txt
```

Want to learn more? Review the documentation on the git restore command.

Unstaging is safer than deleting and allows you to review and edit before finalizing the commit.

- Use the T git status command to confirm that the **hello.txt** file appears under *Changes not staged for commit*.

Running the command *git restore --staged [file]* undoes a *git add* without discarding your work. This is useful when you staged something too early or want to split changes into multiple commits.

Open the **hello.txt** file in your editor to verify that your added line (*I need to add 1 more thing*) is still there.

- Stage the T hello.txt file again by using the T git add command.
- Unstage the changes to the T hello.txt file by running the following command:

```
git restore --staged hello.txt
```

- Discard the change to the T hello.txt file completely by using the T git restore command.

**Expand this hint for guidance on discarding changes to a file.**

- Run the following command to unstage changes:

```
git restore hello.txt
```

Use *git status* to confirm:

*nothing to commit, working tree is clean*

- Display the contents of the T hello.txt file by using the T cat command to verify that the added line "I need to add 1 more thing" is no longer there.

**Expand this hint for guidance on the cat command.**

- Run the following command to view the hello.txt file:

```
cat hello.txt
```

Want to learn more? Review the documentation on the cat command.

Alternatively, you can combine both actions in one command if you prefer: *git restore --staged --worktree hello.txt*.

- Open the **hello.txt** file, add T New content here on a new line, and then save the file.
- Stage the T hello.txt file by using the T git add command.
- Commit the changes by using the T git commit -m command and T "Update content in hello.txt" as the message.
- Confirm the change by using the T git log command.

You should be able to see your most recent commit in the git log.

- Open the **hello.txt** file again, add T This is an additional update to the file on a new line, and then save the file.
- Stage the T hello.txt file *again* by using the T git add command.
- Amend the most recent commit to include your additional edit by running the following command:

```
git commit --amend -m "Updated hello.txt with additional content"
```

- Confirm the change by using the T git log command.

Your commit will now be updated with the text: *Updated hello.txt with additional content*.

⚠ The *git commit --amend* command is great for fixing the most recent commit–such as correcting typos in the message, adding forgotten files, or making small tweaks–before pushing to GitHub. You

can also use it to discard unwanted experimental changes: revert the file to its previous state (e.g., using *git restore [file]*), stage the reverted version with *git add*, and then amend.

**Important**: Avoid amending commits you've already pushed to a shared branch. It rewrites history, which can cause confusion or conflicts for collaborators working on the same repository.

## Check your work

Verify

# Optimize your Git workflow

## Hints Enabled

No Yes

- Tag your most recent commit–T v1.0–by using the T git tag command and T "First stable version" as the message.

  **Expand this hint for guidance on tagging your most recent commit.**

  - Run the following command to tag your most recent commit:

    ```
    git tag -a v1.0 -m "First stable version"
    ```

Tags make it easy to reference specific points in history, especially for releases or deployments.

Want to learn more? Review the documentation on tagging your most recent commit.

- List the tags by using the T git tag command.

  **Expand this hint for guidance on listing tags.**

  Run the following command to list tags:

  ```
  git tag
  ```

- Edit the T hello.txt file to make a small change by adding the text T Small change on a new line, save the file, and then check the status by running T git status.
- Stash the file without staging or committing it by using the T git stash command.

**Expand this hint for guidance on stashing a file.**

- Run the following command to stash the file:

```
git stash push -m "Small change in hello.txt"
```

Want to learn more? Review the documentation on the git stash command.

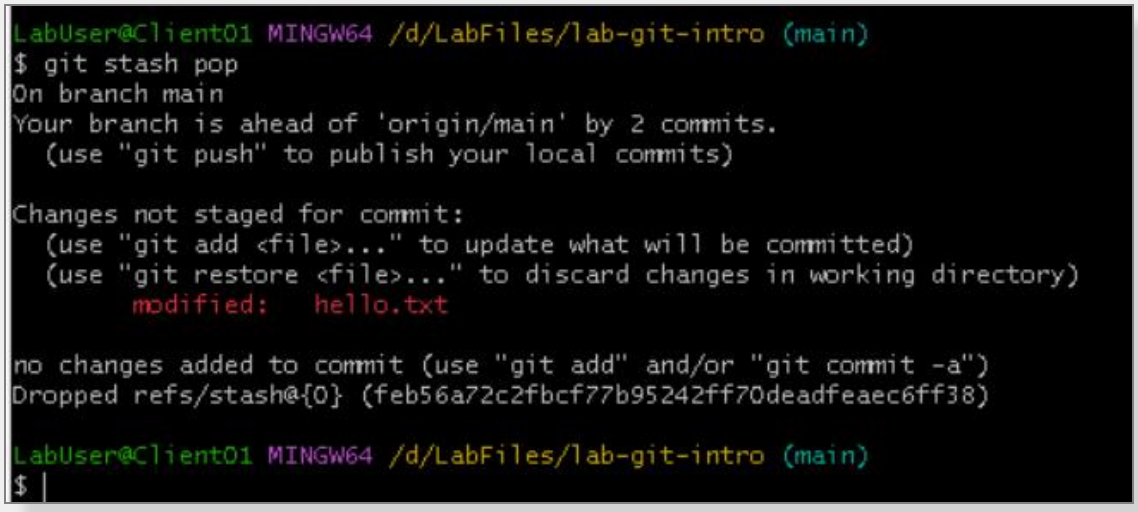
- Confirm the changes were removed by running the T git status command.
- Review your stashed entries by running the T git stash show command.
- Reapply the changes by using the T git stash command with the T pop argument.

**Expand this hint for guidance on reapplying changes.**

- Run the following command to reapply changes:

```
git stash pop
```

- The results should look like the following screenshot:




Git stash is great for pausing work in progress when switching tasks or branches.

- Create a short alias named T hist for the T "log --oneline --graph --decorate --all" common log format by using the T git config command.

**Expand this hint for guidance on creating a short alias.**

- Run the following command to create a short alias:

```
git config --global alias.hist "log --oneline --graph --decorate --all"
```

- Run the T git command with the T hist alias.

**Expand this hint for guidance on running a command using an alias.**

- Run the following command to run the command using an alias:

```
git hist
```

Aliases improve your speed and reduce typing for routine Git operations.

- Stage and commit the hello file by using the following command:

```
git commit hello.txt -m "More updates in hello.txt"
```

This git command stages and commits only the changes in the file hello.txt directly to the repository history with the specified message, bypassing the separate *git add* step.

- Push the T main branch by using the T git push command, and sign in to GitHub with your browser.

**Expand this hint for guidance on pushing to GitHub.**

- Run the following command to push to GitHub:

```
git push -u origin main
```

- When prompted, sign in to GitHub by using the **Sign in with your browser** option.
- If prompted, on the *Authorize Git Credential Manager* page, select **Authorize git-ecosystem**.
- Close the **Git Credential Manager** browser tab.

Want to learn more? Review the documentation on pushing to GitHub.

- Create a new tag named T v1.0.0 with T Second stable version as the message by using the T git tag command.

**Expand this hint for guidance on tagging your most recent commit.**

- Run the following command to tag your most recent commit:

```
git tag -a v1.0.0 -m "Second stable version"
```

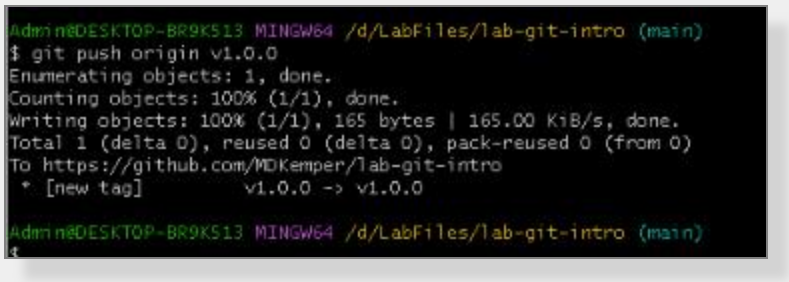
- Push the **v1.0.0** tag by using the T git push command.

**Expand this hint for guidance on pushing to GitHub.**

- Run the following command to push to GitHub:

```
git push origin v1.0.0
```

- The results should look like the following screenshot:




## Check your work

Verify

# Summary

Congratulations, you have completed the **Track Local Changes and Manage Commits** Challenge Lab.

You have accomplished the following:

- Configured your environment and starter project.
- Staged and saved changes.
- Reversed changes and amended commits.
- Optimized your Git workflow.

## Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

⚠ Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

## Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to Challenge Labs feedback.

## Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Track Local Changes and Manage Commits [Guided]
- Create Branches and Merge Changes Safely [Guided]
- Collaborate Remotely and Use Pull Requests [Guided]
- Build a Real Project by Using Git Workflows [Advanced]
- Write Code with GitHub Copilot Suggestions [Guided]

- Control Git Behavior with Config and Ignore Files [Guided]
- Rebase, Stash, and Apply Commits Precisely [Guided]
- Manage Tasks by Using GitHub Issues and Boards [Guided]
- Automate Testing and CI with GitHub Actions [Guided]
- Review and Refactor Code with Copilot [Guided]
- Implement Release Management and Versioning in GitHub [Expert]